Design Pattern Remote Facade e Data Transfer Object

Strategie di distribuzione

Questa sezione affronta il problema di come posizionare le classi di un'applicazione (es. Clienti, Ordini, Prodotti) sui vari host. La considerazione fondamentale è la **performance**: una chiamata di metodo tra processi su host diversi è drasticamente più lenta di una chiamata locale, cioè interna allo stesso processo (circa 20.000 ns contro 1 ns).

Il problema della granularità

Se si distribuiscono i singoli oggetti (strategia basata sulle classi), si finisce per avere troppe chiamate remote. Gli oggetti locali tendono ad avere interfacce fine-grained (molti metodi piccoli), mentre per l'accesso remoto è necessario combinare più operazioni in interfacce coarse-grained (pochi metodi che fanno molto) per ridurre il traffico di rete.

La strategia efficace

Invece di spargere le classi su macchine diverse, è preferibile inserire le classi in un singolo processo e poi replicare quel processo su più host. Questo permette di mantenere le interazioni locali fine-grained all'interno del processo, minimizzando i confini di distribuzione.

Dove separare

I punti naturali in cui è possibile o consigliabile dividere un sistema in processi distinti sono i seguenti:

- **Client e server**: la separazione classica dove i desktop (client) condividono dati presenti su un repository centrale.
- **Server e database**: una separazione quasi sempre presente. Anche se le stored procedures permetterebbero di eseguire logica nel processo del database, spesso non è pratico (non è comodo, portabile, mantenibile).
- **Web server e application server**: possono essere separati (uno gestisce HTTP, l'altro la logica), anche se in molti casi risiedono nello stesso processo.
- **Software di terze parti**: package forniti da vendor diversi potrebbero dover necessariamente girare su processi distinti.
- **Nota generale**: si consiglia di separare la parte server in più processi solo se esistono validi motivi specifici. Questo perché ogni separazione ha un certo costo in termini di complessità, latenza, gestioni errori, ecc.

Come separare

Questa sezione descrive i design pattern da utilizzare per gestire i confini della distribuzione una volta definiti:

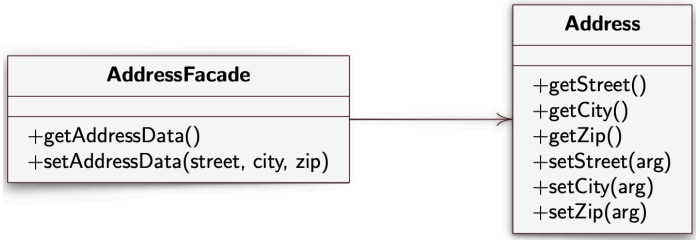
- **Limitare i confini**: l'obiettivo è avere meno confini di distribuzione possibile.

- **Interno contro esterno**: all'interno dello stesso host si continuano ad usare oggetti fine-grained (piccoli e modulari). Ai confini della distribuzione si posizionano oggetti coarse-grained.
- **Remote Facade**: gli oggetti di confine agiscono da Facade (facciata). Il loro unico obiettivo è fornire un'interfaccia remota (esponendo pochi metodi grossi, coarse-grained) che delega il lavoro agli oggetti interni (fine-grained), nascondendo la complessità delle chiamate remote.
- **Data Transfer Object** (DTO): poiché le chiamate remote sono costose, non si possono passare singoli oggetti o fare tante chiamate per recuperare i dati. Si usano i DTO per trasferire grandi quantità di dati serializzati in un'unica chiamata.

Remote Facade

Intento e problema

Il pattern Remote Facade ha l'intento di fornire un'interfaccia a grana grossa (coarse-grained) su oggetti a grana fine (fine-grained) per migliorare l'efficienza delle comunicazioni in rete.



Nasce dal problema che le chiamate inter-processo (remote) sono molto più costose e lente di quelle locali (a causa di latenza, controlli di sicurezza e serializzazione). Se si esponessero direttamente gli oggetti locali (che tipicamente hanno molti metodi piccoli, come getCity(), getZip()), il client dovrebbe effettuare troppe chiamate remote, degradando le prestazioni e complicando il codice.

Soluzione e funzionamento

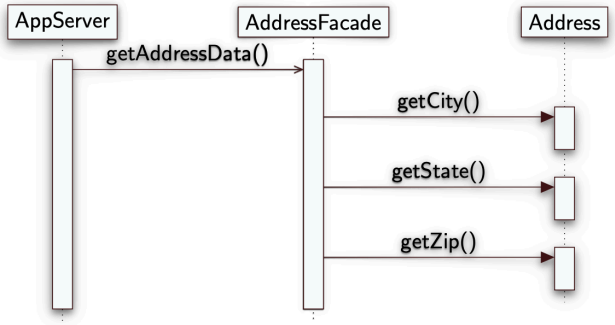
La soluzione consiste nell'introdurre un oggetto Remote Facade che agisce come una facciata per gli oggetti locali:

- **Interfaccia coarse-grained**: sostituisce molti metodi piccoli (get/set) con pochi metodi "*bulk accessors*" che trasferiscono grandi quantità di dati in una sola volta (ad esempio getAddressData() invece di tanti getter singoli).
- **Traduzione**: quando il client invoca un metodo sul Facade, quest'ultimo esegue localmente le numerose chiamate necessarie sugli oggetti del dominio e restituisce il risultato aggregato.
- **Separazione**: il Remote Facade gestisce la distribuzione ma non deve contenere logica di dominio.
- **Responsabilità aggiuntive**: i metodi del Facade sono il punto ideale per gestire la sicurezza (Access Control List, per verificare se gli utenti possono chiamare certi metodi) e le transazioni (avvio e commit della transazione per ogni chiamata).

Esempio Remote Facade

Esempio concettuale (Address)

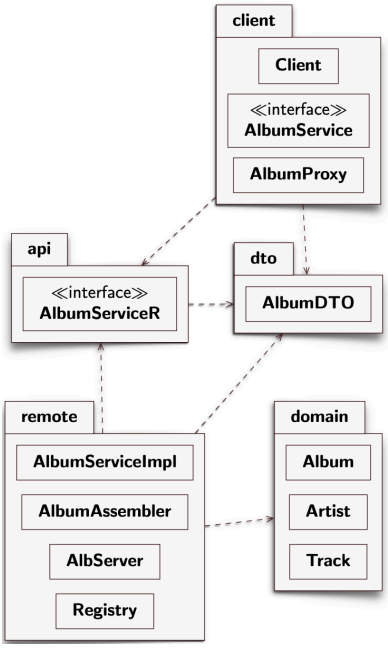
- Esiste un oggetto fine-grained chiamato Address con metodi singoli come getStreet(), getCity(), getZip().
- Viene introdotto un AddressFacade che espone un metodo coarse-grained getAddressData().
- Quando il server riceve la chiamata getAddressData(), il facade chiama localmente i vari getter di Address, impacchetta i dati e li restituisce in un'unica risposta.



Esempio implementativo (Album Service)

Questo esempio rappresenta un'architettura distribuita per la gestione di album musicali basata su RMI (Remote Method Invocation):

- **Interfaccia**: AlbumServiceR è l'interfaccia del Remote Facade che Remote estende e definisce i metodi disponibili al client (es. getAlbum, createAlbum).
- **Implementazione**: la classe **AlbumServiceImpl** è il vero e proprio Remote Facade lato server. Implementa l'interfaccia e gestisce le richieste in arrivo dalla rete.
- **Funzionamento**: AlbumServiceImpl riceve le chiamate remote, interagisce con gli oggetti del dominio (Album, Artist, Track), usa un assemblatore (AlbumAssembler) per convertire i dati in oggetti di trasferimento (DTO) e risponde al client.
- **Setup**: una classe AlbServer crea l'istanza del RemoteFacade (AlbumServiceImpl) e la registra nel servizio di Naming per renderla rintracciabile dai client.



Data Transfer Object

Intento e definizione

Il Data Transfer Object è un pattern il cui intento è creare un oggetto che trasporti dati tra processi differenti, con l'obiettivo principale di ridurre il numero di chiamate a metodo necessario

Il **problema** è che quando si lavora con interfacce remote (come il Remote Facade), ogni singola chiamata attraverso la rete è costosa in termini di prestazioni (latenza, overhead).

→ se si usassero singoli metodi per ogni dato, il traffico sarebbe eccessivo.

→ passare tanti parametri in un unico metodo è scomodo e poco gestibile.

→ spesso il valore di ritorno di un metodo è limitato a un singolo oggetto/valore.

La **soluzione** consiste nel creare un oggetto specifico (il DTO) progettato per contenere tutti i dati necessari per una specifica chiamata. Questo oggetto deve essere serializzabile per poter essere trasmesso attraverso la rete.

Funzionamento

Il funzionamento di un DTO è il seguente:

- **Struttura**: è un semplice contenitore di dati con vari campi e i relativi metodi di accesso (getter e setter).
- **Contenuto**: i campi sono solitamente tipi primitivi, classi semplici (come String o Date) o altri DTO. La struttura deve rimanere semplice. Spesso un singolo DTO aggrega dati provenienti da più oggetti del dominio lato server.
- **Specificità**: il DTO è spesso progettato su misura per un particolare client (ad esempio, può contenere esattamente i dati necessari per riempire una specifica pagina web).

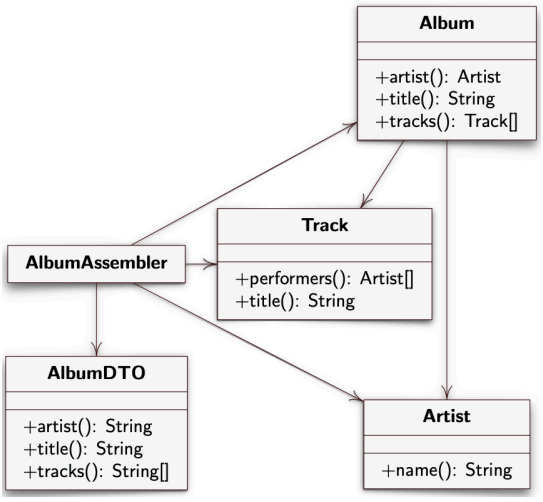
Serializzazione

Il DTO è responsabile della propria serializzazione per viaggiare sulla rete:

- **Formato**: dipende dalla tecnologia (es. binario in Java, XML, JSON).
- **Fragilità**: la serializzazione binaria nativa di Java può rendere la comunicazione fragile (se server e client hanno versioni diverse della classe, si verifica inevitabilmente un errore). La serializzazione in formato testuale (come XML) tende ad essere più tollerante ai cambiamenti.

Assembler

Per trasferire i dati tra i **Domain Objects** (gli oggetti che contengono la logica di business sul server) e i **DTO**, si utilizza spesso un oggetto chiamato **Assembler**. Questo componente si occupa di mappare e copiare i dati dall'uno all'altro, mantenendo separati i due mondi.



Implementazione Remote Facade

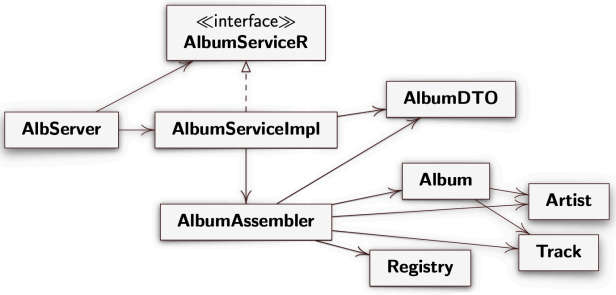
L'implementazione del pattern Remote Facade prevede la creazione di un oggetto che fornisca una "facciata" a grana grossa (coarse-grained) per ricevere le chiamate attraverso la rete.

- **Ruolo**: nel caso di studio presentato, la classe che svolge questo ruolo è AlbumServiceImpl (lato server). Essa riceve le richieste remote e delega il lavoro agli oggetti del dominio (come Album, Artist, Track) e agli assemblatori.
- **Tecnologia**: sebbene a basso livello le comunicazioni viaggino su socket, per semplificare l'implementazione si utilizza RMI (Remote Method Invocation), che gestisce la complessità della rete.
- **Lato server**: una classe apposita (es. AlbServer) contiene il main, crea l'istanza di AlbumServiceImpl e la registra nel servizio di Naming per renderla accessibile.

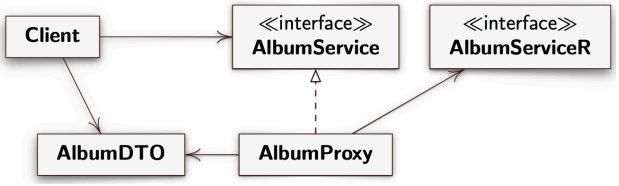
Remote Method Invocation

RMI è il meccanismo Java utilizzato per permettere la comunicazione tra client e server.

- **Interfaccia comune**: per far comunicare le due parti, si definisce un'interfaccia Java condivisa (nell'esempio AlbumService o AlbumServiceR). Questa interfaccia deve estendere `java.rmi.Remote`.
- **Eccezioni**: ogni metodo definito nell'interfaccia e nella sua implementazione (AlbumServiceImpl) deve dichiarare di poter lanciare un'eccezione di tipo RemoteException, necessaria per gestire gli errori di rete.
- **Funzionamento**:
  - **lato server**: si crea l'oggetto e lo si registra su un **RMI registry** (servizio di Naming).



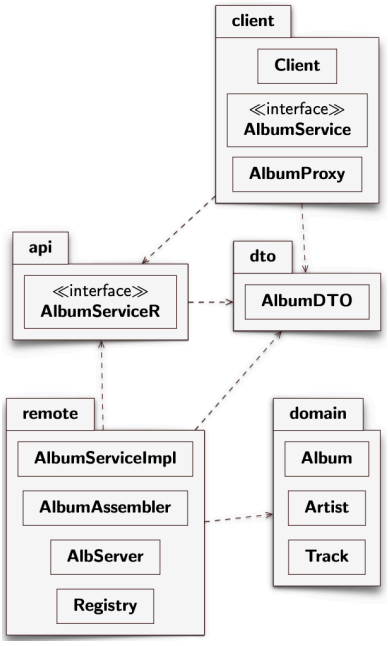
- **lato client**: si utilizza un'operazione di **lookup** sul registro RMI tramite un identificatore per localizzare l'oggetto remoto e ottenerne un proxy per effettuare le chiamate.



Esempio di distribuzione

Le classi e i package vengono distribuiti fisicamente tra i due host (client e server) nell'esempio dell'applicazione musicale, nel seguente modo:

- **Host client** → contiene i package:
  - **client**: logica dell'applicazione client (es. la classe Client con il main).
  - **api**: interfacce condivise.
  - **dto**: i data transfer object per i dati.
- **Host server** → contiene i package:
  - **api** e **dto**: (condivisi con il client).
  - **remote**: implementazione del Remote Facade (AlbumServiceImpl, AlbServer).
  - **domain**: logica di business (Album, Artist, Track).



Componenti chiave nella distribuzione

- **AlbumProxy** (lato client): è un componente che scherma il client dalla complessità di RMI. Il client chiama metodi su questo proxy locale, il quale gestisce le eccezioni di rete (RemoteException) e invoca il vero servizio remoto, rendendo l'uso del servizio più trasparente per il codice client.
- **AlbumAssembler**: utilizzato lato server per convertire gli oggetti del dominio in DTO prima che attraversino la rete.